AI Development Expert – Final Project

Preliminary Report

NegoPlay

Strategic Profiles from Bridge to Business:

An Empirical Investigation of Decision-Making Styles

using LLM Agents

Student: Anna Ben-Shushan

Supervisor: Dr. Yoram Segal

Course: AI Development Expert

Document: Preliminary Project Report

Version: 1.0

Date: May 30, 2026

Acknowledgements

I would like to thank **Dr. Yoram Segal**, my project supervisor, for his structured methodology framework – in particular the requirement of clearly identified anchor papers, single-SDK architecture, and explicit baseline comparisons. These principles shaped this project from the first day.

This project is also the empirical baseline for my PhD research at LUT University (Finland), under the supervision of **Prof. Jari Hämäläinen**. I gratefully acknowledge **Dr. Nezer Jacob Zaidenberg**, my PhD supervisor and advisor on bridge and cybersecurity matters, for ongoing discussions throughout this work.

The 149,208-row dataset used in this work was collected from the public EuroBridge database (European Bridge League, 2016–2025) and processed using a custom Python pipeline developed during the first weeks of 2026.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and Motivation

Decision making under incomplete information is one of the hardest problems in artificial intelligence. While modern AI has reached super-human performance in fully observable games such as chess (Silver et al., 2017), Go, and Atari (Schrittwieser et al., 2020), real-world strategic environments – business negotiation in particular – are characterized by hidden information, partial communication, and a mix of cooperation and competition that is hard to model directly.

Bridge is a unique testbed for these challenges. Each of the four players sees only 13 of the 52 cards, must cooperate with a partner while competing against two opponents, and communicates through a highly structured bidding language with approximately 10^{47} possible sequences. Bridge bidding is, formally, a multi-agent negotiation under imperfect information with measurable outcomes. Rong et al. (2019) showed that a deep learning system splitting bidding into estimation (ENN) and policy (PNN) networks can beat the world-leading rule-based engine Wbridge5; Brown and Sandholm (2019) demonstrated that imperfect-information AI can defeat elite humans in multi-player poker.

NegoPlay asks a more applied question: *can we extract decision-making profiles from a large dataset of elite bridge play, and reuse those profiles to predict and simulate behavior in business negotiation scenarios?*

1.2 Why this matters

Business negotiation suffers from a fundamental data scarcity problem. Real negotiation transcripts are proprietary, outcomes are rarely measurable in standardized form, and no public datasets allow training of behavioral models. Bridge, in contrast, offers a clean laboratory:

- More than 149,000 structured decision points with measurable outcomes.
- Clear win/loss signals (contract success, IMP scores, VP conversion).
- Tournament-grade decisions by expert players, with explicit player identity per seat.

If profiles discovered in this clean environment transfer measurably to the noisier domain of negotiation, this opens a methodological path for studying business strategy without proprietary negotiation data.

1.3 Literature Review and Positioning

This section grounds NegoPlay in the wider research conversation. Following Lockett et al. (2007), we treat an unknown counterpart as a mixture of *cardinal strategies* that can be discovered and re-used. We organize the literature into five themes.

Theme 1: Player-style clustering from gameplay sequences

Our methodological anchor is Talwadker et al. (2022), who introduced **CognitionNet**. Using a sequence-to-sequence interpreter coupled to a classifier, they discovered behavioral profiles directly from raw Rummy telemetry. NegoPlay replaces "Rummy telemetry" with "bridge bidding sequences" but keeps the same logic: *styles emerge from sequences of actions, without predefined rules.* Sztyber-Betley et al. (2023) provide an alternative path – learning a continuous 8-dimensional embedding of bridge hands – that we may incorporate in a second iteration.

Theme 2: Inference + decision split (theory of mind)

Our second anchor is Rong et al. (2019), whose ENN/PNN architecture separates the inference of partner's hidden cards from the action policy. Wang et al. (2024) extend this with a conditional GAN inference module. NegoPlay applies the same split at the LLM level: the cluster profile is the *inference* ("what kind of agent am I facing?"), and the system prompt is the *policy* ("how do I act?"). This theory-of-mind-before-action structure is now standard in published bridge AI.

Theme 3: Opponent as mixture of cardinal styles

Lockett et al. (2007) is the theoretical foundation for the entire project. Their result – that explicit mixture coefficients over a small set of cardinal opponent types outperform monolithic networks – justifies the central NegoPlay premise: every negotiation counterpart can be treated as a weighted mixture of the few profiles discovered in the bridge data.

Theme 4: Bridging structured games and natural-language negotiation

James et al. (2021) encode both actions and "cheap talk" messages into unified attitude vectors, then cluster them. This is the conceptual glue between the bridge stage (structured signals) and the negotiation stage (free-form text), and it is the paper we cite when defending the transfer claim.

Theme 5: Existing bridge AI engines as benchmarks

Yeh et al. (2018) and Kita et al. (2024) train deep RL bidders that beat Wbridge5; the open-source BEN engine (lorserker, 2023) provides a neural-network-based, scriptable opponent that we can use to ground LLM-vs-LLM games against an objective player. Jarosz (2022) provides a clean survey of the field.

What others did, and what NegoPlay does differently

Table 1.1: NegoPlay positioning against representative prior work.

Reference	**What they did**	**What NegoPlay does differently**
Talwadker et al. (2022)	Clustered Rummy players from telemetry sequences.	Cluster bridge *bidding* sequences; then *reuse* the profiles in a second, different domain (negotiation).
Rong et al. (2019)	Built a deep ENN/PNN bidding system that beats Wbridge5.	Replace the deep networks with *LLM agents* prompted by the discovered profile; trade peak strength for explainability and cross-domain reuse.
Lockett et al. (2007)	Evolved explicit opponent models using NEAT.	Replace the evolved cardinal set with a *data-driven, statistically validated* cardinal set (K-means + HDBSCAN).
James et al. (2021)	Clustered attitudes from actions + cheap talk in toy games.	Apply the same logic to a real, large-scale, expert dataset – and explicitly test transfer between two domains.

2 Research Question and Objectives

2.1 Primary Research Question

> *Can LLM agents built from automatically-discovered bridge decision-making profiles exhibit behavioral consistency (*Spearman $\rho \geq 0.7$*) between winning in bridge bidding simulations and winning in parallel business negotiation simulations?*

2.2 Hypotheses

H1 – Statistical: K-means clustering on five behavioral features will produce 3–5 stable profiles with Silhouette score ≥ 0.4 and $p < 0.05$ against random clustering.

H2 – Behavioral: LLM agents instantiated with these profiles will exhibit measurably different behaviors in bridge bidding (per-profile win-rate variance $> 15\%$).

H3 – Transfer (main): The win-rate ranking across profiles will correlate $\rho \geq 0.7$ (Spearman) between the bridge and negotiation domains.

Null: No correlation > 0.5 between domains. This too is a valid research finding.

2.3 Objectives

The project has four operational objectives:

1. **Statistical:** discover 3–5 stable decision profiles from $\approx$1,500 elite players.
2. **Linguistic:** translate each profile into a 5–7 trait skill description using an LLM.
3. **Engineering:** build four LLM agents (one per profile) with verifiable, structured output.
4. **Empirical:** run dual simulations (bridge + negotiation) and measure cross-domain alignment.

2.4 Anchor papers

Following Dr. Yoram Segal's methodology framework, every project declares its anchor papers explicitly:

- **Anchor 1 (Stage 1 – clustering):** Talwadker et al. (2022).
- **Anchor 2 (Stage 3 – inference/decision split):** Rong et al. (2019).
- **Theoretical foundation:** Lockett et al. (2007).

3 Methodology

3.1 Overview

NegoPlay is a four-stage hybrid pipeline that combines classical machine learning with large language models (see Figure 3.1).

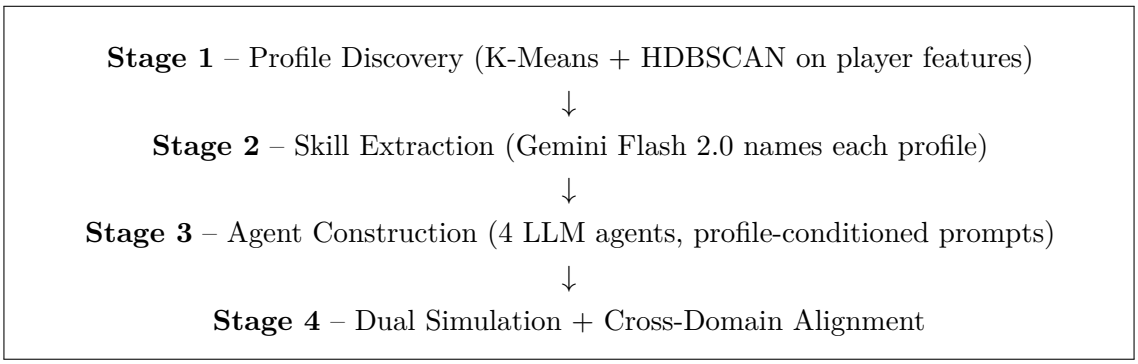


Figure 3.1: The four-stage NegoPlay pipeline. Stages 1 and 4 produce the empirical evidence; stages 2 and 3 produce the agents.

3.2 Dataset

The dataset was collected by the author from the public EuroBridge database between January and May 2026, using a custom Python scraping pipeline.

Table 3.1: NegoPlay dataset summary. Rows are progressively filtered as we narrow from the raw scrape to the analysis cohort actually used by Stage 1.

Metric	**Value**
Total rows (board × room)	149,208
European Championships covered	5 (2016–2025)
Unique players (by name)	≈ 2,500
Player-name coverage (N/S/E/W × open/closed)	99.9 %
Card-holding coverage (52 cards)	54 %
Bidding-sequence coverage	31 % (100 % on Herning 2024 + Poznan 2025)
Analysis cohort – progressive filtering for Stage 1:	
Players with ≥ 20 declared boards (legacy threshold)	1,572
Players with ≥ 50 declared *and* ≥ 50 bidding boards (production)	**563**

Why two thresholds appear. The legacy threshold of 20 boards was used in the v2.0 Stage 1 pipeline (April 2026), which produced 64 "Slam Hunters" out of 1,572 qualifying players. PhD supervisor Dr. Nezer Jacob Zaidenberg identified a sample-size problem: with only 20 boards and a rare-event rate near 5 %, the 95 % confidence interval is too wide to distinguish a true Slam Hunter from random variation. The pipeline was revised to require ≥ 50 boards on *both* the declared and the bidding axis, plus a one-sided binomial test (Eq. (4.11)). The cohort that

enters all numerical results in this report is therefore the **563-player** v2.1 cohort; the 1,572 figure is shown only as a reference for the underlying data scale. The full revision is documented in RESEARCH_INSIGHTS.md, sections Q7.5–Q7.6.

3.3 Stage 1 – Profile Discovery (ML)

Ten behavioural features are computed per player and partitioned into two groups: *outcome features* extracted from the `contract` column (any board the player declared) and *process features* extracted from the `bidding` column (any board where the auction is recorded, regardless of who declared). The process / outcome split follows the methodological anchor of Rong et al. (2019), who separate the inference (auction-process) signal from the action (outcome) signal.

- **Outcome features (declarer only, 5):** `slam_rate` (% of declarations at slam level 6–7); `double_rate` (% of declared contracts that opponents doubled); `avg_level` (mean contract level $\in [1, 7]$); `nt_rate` (% of declarations in NoTrump); `partscore_rate` (% of declarations at level 1–3).
- **Process features (any seat, from full bidding, 5):** `opening_rate` (% of boards where this player opened the auction); `preempt_rate` (% of opening bids at level ≥ 2); `intervention_rate` (% of boards with a competitive bid into the opponents' auction); `penalty_double_rate` (% of boards with a doubling call by this player); `avg_bids_per_board` (mean number of bids per board).

Note on legacy features. An earlier draft used a 5-feature set including `success_rate` and a composite `avg_risk_score`; both were dropped after the variance audit in Chapter 4 showed that `success_rate` provides almost no separating signal at elite level (all elite players succeed at $\approx 80\,\%$) and that `avg_risk_score` is a linear combination of other features and adds no new information.

After variance and correlation filtering (Chapter 4, Eq. (4.2)), the 10 features reduce to a stable 7–8 axis behaviour space depending on the configuration (8 for the production K-Means / extreme-percentile pipeline; 7 for the V3 audit configuration which also drops `partscore_rate`). Features are then standardised via `StandardScaler` (Eq. (4.1)), reduced via PCA (Eq. (4.4)), and assigned to one of five profiles via the extreme-percentile + binomial-test rule (Eq. (4.11)).

3.4 Stage 2 – Skill Extraction (LLM)

For each of the four extreme profiles we sampled five players and sent their actual played boards to **Gemini 2.5 Flash** in chunks of up to 25 boards, with a strict JSON schema. Each call returns 5–7 decision-making skills, each with a confidence label and supporting board references. A profile-specific guidance prompt (`PROFILE_GUIDANCE`) requires at least two skills to lie on the profile's defining axis; without it, an unguided prompt returned the generic "aggressive competitive bidding" for every profile.

Semantic aggregation

Raw skill names vary in wording (e.g. "initiates slam exploration with splinters" vs. "employs splinter bids for slam exploration"). We merge synonymous skills with TF–IDF cosine similarity and Union–Find clustering (merge threshold 0.40; the lower value 0.30 produced false merges such as "control bidding" with "competitive overcalling"). The most frequent raw name in each cluster becomes the canonical skill label.

Empirical validation

The central question is whether the LLM-described profiles are *real* behavioural types or an artefact of the clustering / an LLM hallucination. We therefore measured each profile's defining behaviour directly from the raw bidding and contract data—no LLM, no clustering—and compared it against a Generalist baseline drawn from the same elite population (an *elite-vs-elite* comparison, which makes any separation conservative). Denominators match Stage 1 exactly: declarer-only boards for the contract-level metrics (slam, partscore, NT) and per-board-with-bidding for the Fighter's penalty doubles. Effect size is Cohen's d; significance is a one-sided Mann–Whitney U test. All four profiles separate from the baseline with large effect sizes and statistical significance (Table 3.2).

Table 3.2: Stage 2 empirical validation: each profile's defining behaviour vs. the Generalist baseline (validation sample, five players per profile).

Profile	**Defining metric**	**Ratio**	**Cohen's d**	**p-value**
Fighter	penalty_double_rate	×1.31	2.13	0.016
Insurance Player	partscore_rate	×1.24	3.30	0.004
Slam Hunter	slam_rate	×1.37	2.81	0.004
NT Specialist	nt_rate	×1.27	4.62	0.004

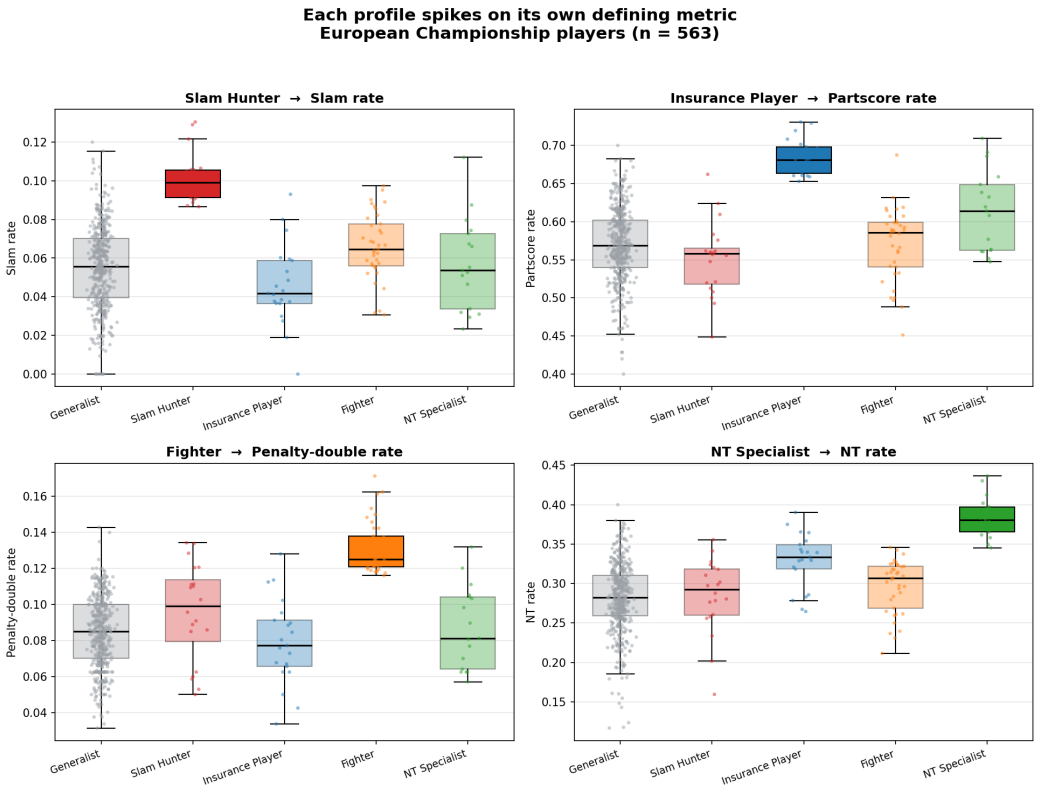
Cohen's $d \geq 0.8$ is conventionally "large" and $d \geq 2.0$ "very large"; all four profiles clear $d = 2.0$ and all are significant at $p < 0.05$. (These ratios use the five-player validation sample and so differ slightly from the full-population ratios in Table **??**; both tell the same story.) A second independent review by a bridge-domain expert confirmed the result with the verdict "proceed to Stage 3". The human-readable profile names are *Slam Hunter*, *Insurance Player*, *Fighter*, and *NT Specialist*.

Methodological note

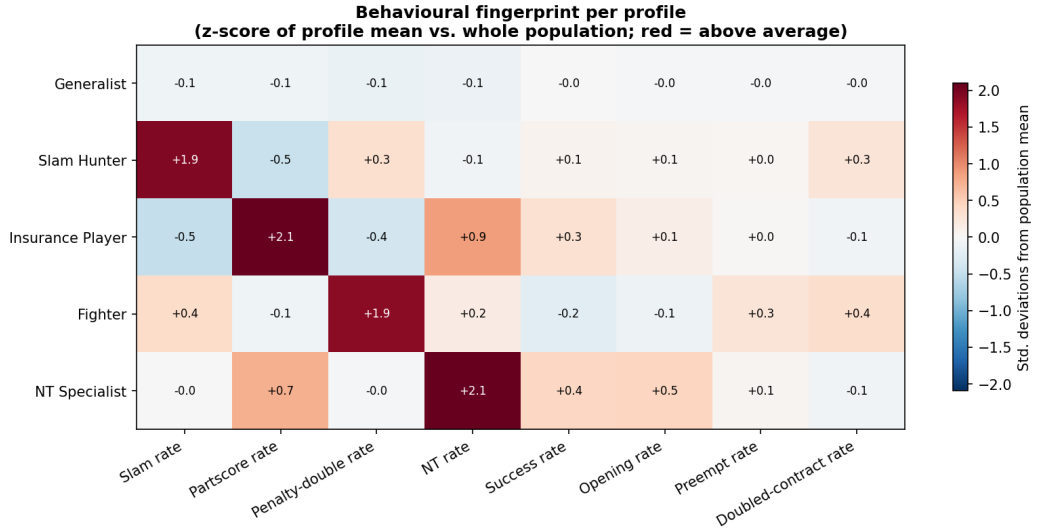
The Fighter metric was initially measured per call (doubles ÷ total calls), which gave a weak, overlapping signal and was rejected on expert review. Switching to the Stage-1-aligned per-board metric (boards with at least one double ÷ boards with bidding) produced the clean ×1.31 separation above. The validation denominator must match the clustering denominator exactly, or a genuine profile can appear spurious.

Visual summary

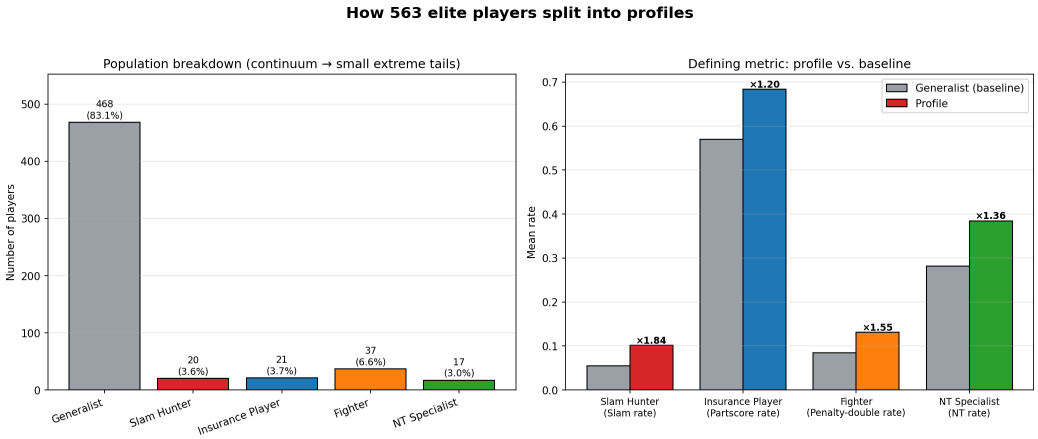
Figure 3.2 presents the validation visually: each profile spikes on its own defining metric (panel a), the per-profile "fingerprint" heatmap shows the selected axis plus coherent side-behaviours we did *not* select for (panel b), and the population splits into a dense Generalist continuum with the four profiles forming the behavioural tails (panel c).



(a) Each profile spikes on its defining metric.



(b) Per-profile behavioural fingerprint (z-scores vs. the population).



(c) Population breakdown and defining-metric ratios.

Figure 3.2: Stage 2 validation visualised (563 players). Generated by

3.5 Stage 3 – Agent Construction

Each profile becomes an LLM agent through a structured system prompt that contains: identity (profile name), 5–7 traits, domain context (bridge *or* negotiation), output schema (JSON), and 1–2 few-shot examples. Two agent classes are implemented:

- `BridgeAgent.make_bid(hand, auction)` – returns a valid bridge bid.
- `NegotiationAgent.respond_to_offer(scenario, history, offer)` – returns a counter-offer plus reasoning.

3.6 Stage 4 – Dual Simulation and Alignment

4a Bridge: 50 randomly generated hands are played by combinations of profile agents; results are scored in IMPs. As a non-LLM benchmark, profile agents can also play one side of a board against the open-source BEN engine (lorserker, 2023) sitting on the other side.

4b Negotiation: four scenarios (M&A, JV equity split, B2B SaaS pricing, government tender) are run for each pair of agents, with up to 10 rounds and an explicit walk-away condition.

4c Alignment: per-profile win rates are computed for both domains; the cross-domain alignment is measured by *Spearman* correlation. A scatter plot is produced; significance is tested via a two-sided permutation test.

3.7 Tools and Stack

Development environment

- **Language:** Python 3.11 in an isolated `venv` virtual environment.
- **Operating system:** Windows 11 with WSL2 for Unix-style tooling.
- **Primary IDE:** Google Antigravity (agent-first development).
- **Secondary IDE:** Microsoft Visual Studio Code.
- **AI coding assistant:** Anthropic Claude Code (CLI), with custom slash commands (e.g. `/bridge-expert`) defined in `.claude/commands/`.
- **Source control:** Git + GitHub, public repository.

Data science and statistical libraries

- **Data manipulation:** `pandas`, `numpy`.
- **Machine learning (scikit-learn):** `StandardScaler`, `RobustScaler` for Z-score normalisation; `PCA` for dimensionality reduction; `KMeans`, `HDBSCAN`, `GaussianMixture` for clustering; `TSNE` for visualisation; `EmpiricalCovariance` for Mahalanobis-distance outlier detection.

- **Statistics:** `scipy.stats.binomtest` (one-sided binomial test for profile assignment); `scipy.stats.chi2` (Mahalanobis threshold).
- **Future stages:** `gensim` (Word2Vec for bidding sequences), `shap` (XAI for cluster explanation).

Visualisation and reporting

- **Plots:** `matplotlib` – PCA scatter, t-SNE scatter, radar charts, scree plots, feature bar charts.
- **Spreadsheets:** `openpyxl` – transparent export of normalised features and PCA loadings to `results/preprocessing.xlsx`.
- **This report:** LaTeX (TeX Live), `natbib`, `booktabs`, `tabularx`.
- **Internal documentation:** Markdown for PRD, RESEARCH_INSIGHTS, CLAUDE.md, and README files.

LLM providers (provider-neutral architecture)

- **Default:** Google Gemini Flash 2.0 – cost \$0.075 / M input, \$0.30 / M output tokens.
- **Cross-validation:** Anthropic Claude (Haiku / Opus) and OpenAI GPT, routed through a single `shared/llm_client.py` wrapper. Running the same prompt across providers strengthens the claim that findings are not a single-provider artefact.
- **Configuration:** all API keys loaded from `.env` via `python-dotenv`; never hard-coded.
- **Retry handling:** `tenacity` for exponential back-off under rate limits.

Testing and quality

- **Testing:** `pytest` (73 unit tests passing across feature engineering, profile assignment, and the bridge-expert validator); `pytest-cov` for coverage reporting.
- **Linting and formatting:** `ruff`.
- **Type validation:** `pydantic` for structured LLM outputs.

Experiment tracking and reproducibility

- **Per-call logs:** JSON-Lines records in `results/llm_logs/` with provider, model, prompt tokens, completion tokens, latency, and cost.
- **Random seeds:** fixed `random_state=42` across scikit-learn calls for reproducible clustering and PCA.
- **Pipeline replay:** the entire Stage 1 pipeline runs end-to-end from a clean clone with a single command via `src/sdk.py`.

Source data

- **EuroBridge public database** – five European Championships (2016–2025).
- **Custom Python scraper** – `requests` + `BeautifulSoup4`, developed in the parent collectBridgeData project.

4 Technical Architecture and Mathematical Foundations

This chapter formalises the algorithms and architectures used by NegoPlay. It is written for technical reproducibility: any reader with the published dataset and the equations below should be able to reproduce every numerical result in this report.

4.1 System Architecture

NegoPlay is a four-stage hybrid pipeline (ML → LLM → LLM → LLM) connected by file artefacts on disk. Figure 4.1 shows the system architecture with stages, intermediate artefacts, and the direction of information flow.

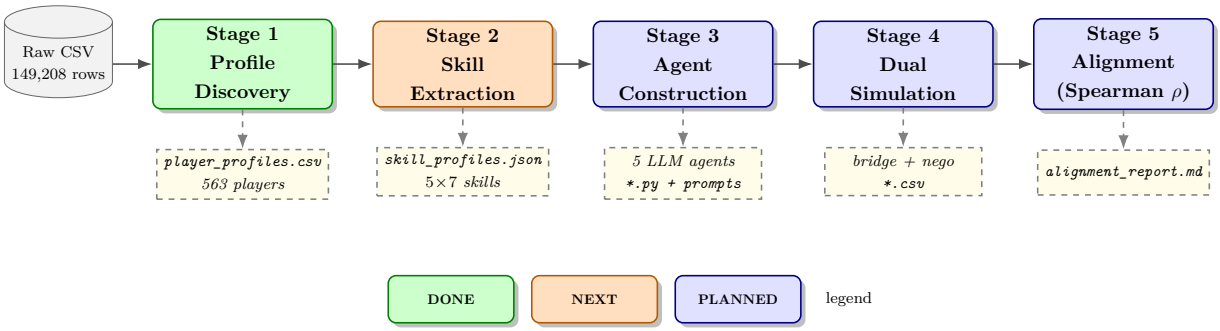


Figure 4.1: NegoPlay system architecture. Solid arrows: information flow between stages. Dashed arrows: persisted artefacts on disk. Stage 1 is complete; Stage 2 is next; Stages 3–5 are planned.

4.2 Stage 1: Mathematical Pipeline

Stage 1 transforms the raw EuroBridge CSV into a labelled profile assignment for each qualifying player. Figure 4.2 shows the full preprocessing and assignment pipeline.

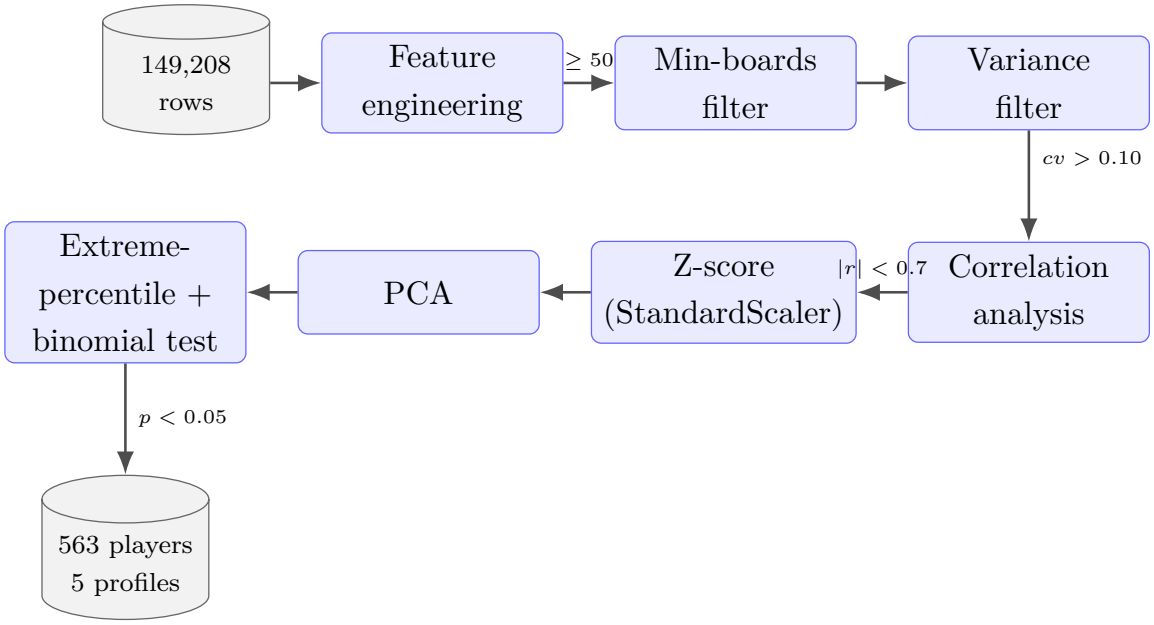


Figure 4.2: Stage 1 pipeline: raw data → feature engineering → statistical filters → scaling → PCA → extreme-percentile profile assignment. Each filter is labelled with its threshold.

4.2.1 Notation

Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ denote the player-feature matrix where $n = 563$ is the number of qualifying players (those with ≥ 50 declared boards *and* ≥ 50 bidding boards). The i-th row $\mathbf{x}_i$ represents one player; the j-th column $\mathbf{X}_{\cdot j}$ represents one feature.

Two configurations of p. The feature count depends on the pipeline:

- **Production pipeline ($p = 8$):** we start from the 10 features of Section 3, then drop `avg_level` (CV = 0.04) and `avg_bids_per_board` (CV = 0.08) for failing the variance gate of Eq. (4.2).
- **V3 audit configuration ($p = 7$):** same as production, but `partscore_rate` (CV = 0.09) is also dropped as borderline-low-variance during Dr. Rami's preprocessing audit. The Mahalanobis-distance outlier detection below is run in the V3 configuration, hence $p = 7$ in Eq. (4.10).

4.2.2 Z-score standardisation

To make features comparable across different scales (e.g., `slam_rate` $\in [0, 0.15]$ vs. `nt_rate` $\in [0, 0.45]$), each feature is standardised to zero mean and unit variance:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{4.1}$$

Parameters:

z_{ij} standardised score of player i on feature j.

x_{ij} raw value of feature j for player i (e.g., player's observed `slam_rate`).

μ_j population mean of feature j across all $n = 563$ qualifying players: $\mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}$.

σ_j population standard deviation of feature j: $\sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \mu_j)^2}$.

Equation (4.1) is implemented by `sklearn.preprocessing.StandardScaler`. A z-score of $+2$ means the player is two population standard deviations above the mean on that feature; in a normal distribution this corresponds to the $\approx 97.5^{\text{th}}$ percentile. We use the same formula both for preprocessing (input to K-Means / PCA) and for profile-significance scoring (column `profile_z` in `player_profiles.csv`).

4.2.3 Variance audit (coefficient of variation)

Following Dr. Rami's preprocessing review, we audit each feature by its coefficient of variation:

$$\mathrm{CV}_j = \frac{\sigma_j}{|\mu_j|} \tag{4.2}$$

Parameters:

CV_j coefficient of variation of feature j (dimensionless).

σ_j population standard deviation of feature j (as in Eq. (4.1)).

μ_j population mean of feature j (as in Eq. (4.1)).

Features with $\mathrm{CV}_j < 0.10$ are discarded as low-information: their relative spread is below 10% of the mean, so they cannot meaningfully distinguish players. In our data, `avg_level` ($\mathrm{CV} = 0.04$) and `avg_bids_per_board` ($\mathrm{CV} = 0.08$) were removed by this filter.

4.2.4 Principal Component Analysis

PCA computes the eigendecomposition of the standardised covariance matrix:

$$\mathbf{C} = \frac{1}{n-1}\,\mathbf{Z}^{\top}\mathbf{Z} \tag{4.3}$$

Parameters:

$\mathbf{C}$ $p \times p$ covariance matrix of the standardised features.

$\mathbf{Z}$ $n \times p$ matrix of z-scores from Eq. (4.1) ($n = 563$, $p = 8$).

$n - 1$ degrees-of-freedom correction (Bessel's correction).

Solving the eigenvalue problem

$$\mathbf{C}\,\mathbf{v}_k = \lambda_k \mathbf{v}_k, \qquad k = 1, \ldots, p \tag{4.4}$$

Parameters:

$\mathbf{v}_k$ k-th eigenvector (principal direction in feature space).

λ_k k-th eigenvalue, ordered $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$.

k component index, $k \in \{1, \ldots, p\}$.

The fraction of variance explained by component k is

$$\mathrm{EV}_k = \frac{\lambda_k}{\sum_{i=1}^{p}\lambda_i} \tag{4.5}$$

We retain the top K components that satisfy $\sum_{k=1}^{K} \mathrm{EV}_k \geq 0.80$ (cumulative variance threshold). In our data, $K = 5$ components achieve 85.2% cumulative variance: $\mathrm{EV} = (0.246, 0.191, 0.135, 0.103, 0.077)$.

4.2.5 Clustering validity: Silhouette score

For each player i assigned to cluster $C_{k(i)}$, define:

$$a(i) = \frac{1}{|C_{k(i)}| - 1} \sum_{\substack{j \in C_{k(i)} \\ j \neq i}} d(\mathbf{x}_i, \mathbf{x}_j) \tag{4.6}$$

$$b(i) = \min_{\ell \neq k(i)} \frac{1}{|C_\ell|} \sum_{j \in C_\ell} d(\mathbf{x}_i, \mathbf{x}_j) \tag{4.7}$$

Parameters:

- $a(i)$ mean Euclidean distance from player i to all *other* players in their own cluster $C_{k(i)}$ – the "cohesion" term.
- $b(i)$ mean Euclidean distance from player i to the nearest *other* cluster – the "separation" term. The min ranges over all clusters $\ell \neq k(i)$.
- $d(\mathbf{x}_i, \mathbf{x}_j)$ Euclidean distance in the scaled PCA space.
- $C_{k(i)}$ the cluster to which player i is assigned.

The silhouette score for player i and the overall mean are:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad \bar{s} = \frac{1}{n} \sum_{i=1}^{n} s(i) \tag{4.8}$$

Range: $s(i) \in [-1, +1]$ for every player. Interpretation:

- $\bar{s} \geq 0.50$ strong cluster structure.
- $0.25 \leq \bar{s} < 0.50$ weak structure.
- $\bar{s} < 0.25$ no meaningful cluster structure.

Our maximum across five preprocessing configurations was $\bar{s} = 0.24$ (V2: 8 features + PCA(3), Z-score). Computed via `sklearn.metrics.silhouette_score`.

4.2.6 Mahalanobis-distance outlier detection

For multivariate outlier detection prior to clustering (V3 configuration only), we use the squared Mahalanobis distance:

$$D^2(\mathbf{x}_i) = (\mathbf{x}_i - \boldsymbol{\mu})^\top \mathbf{S}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) \tag{4.9}$$

Parameters:

- $D^2(\mathbf{x}_i)$ squared Mahalanobis distance of player i from the data centre.
- $\mathbf{x}_i$ feature vector of player i (after scaling).
- $\boldsymbol{\mu}$ sample mean vector of all players, $\boldsymbol{\mu} \in \mathbb{R}^p$.

$\mathbf{S}^{-1}$ inverse of the empirical covariance matrix (estimated by `sklearn.covariance.EmpiricalCovarian`). The inverse weights features by their inter-correlations, so a player who is unusual on *correlated* features counts as more outlying.

Under multivariate normality, $D^2 \sim \chi^2_p$, so player i is flagged as an outlier when

$$D^2(\mathbf{x}_i) > \chi^2_{p,\,1-\alpha} \tag{4.10}$$

Parameters:

$\chi^2_{p,1-\alpha}$ $(1-\alpha)$ critical value of the chi-square distribution with p degrees of freedom (number of features). Computed via `scipy.stats.chi2.ppf(1-`α`, df=`p`)`.

α significance level; we use $\alpha = 0.01$ (top 1% of the tail). For $p = 7$, this gives a threshold of ≈ 18.5.

Methodological note: outlier removal is appropriate for K-Means and GMM but counterproductive for the extreme-percentile method, since the outliers are precisely the profile members we want to identify.

4.2.7 Extreme-percentile profile assignment

The production pipeline uses an extreme-percentile method on the standardised features, supported by a binomial significance test. For each profile axis j (slam_rate, partscore_rate, nt_rate, penalty_double_rate), we:

1. Identify the top 10% of players on $z_{\cdot j}$ (Eq. 4.1).
2. Test the null hypothesis that their observed rate equals the population baseline p_0 using a one-sided binomial test (Eq. 4.11).
3. Assign the player to profile j only if both gates pass.

$$\text{p-value}_i = \sum_{m=k_i}^{n_i} \binom{n_i}{m} p_0^m \,(1-p_0)^{n_i-m} \tag{4.11}$$

Parameters:

n_i player i's denominator: `n_declared` for outcome features (Slam Hunter, Insurance Player, NT Specialist) or `n_bidding_boards` for process features (Fighter).

k_i number of "successes" for player i on the profile axis (e.g., number of slam contracts declared, for Slam Hunter).

p_0 population baseline rate of the event (e.g., $p_0 = 0.055$ for slam).

$\binom{n_i}{m}$ binomial coefficient "n_i choose m".

m summation index over all outcomes at least as extreme as the observed k_i.

Interpretation: the p-value is the probability of observing at least k_i successes under the null hypothesis that the player's true rate equals the baseline p_0 (one-sided, "greater than" alternative). We require p-value$_i < 0.05$ for profile assignment. Implemented as `scipy.stats.binomtest(`k_i`,` n_i`,` p_0`, alternative='greater')`.

4.3 Stage 3: LLM Agent Architecture

Each profile becomes an LLM agent through a structured prompt. Figure 4.3 shows the agent's request/response cycle.

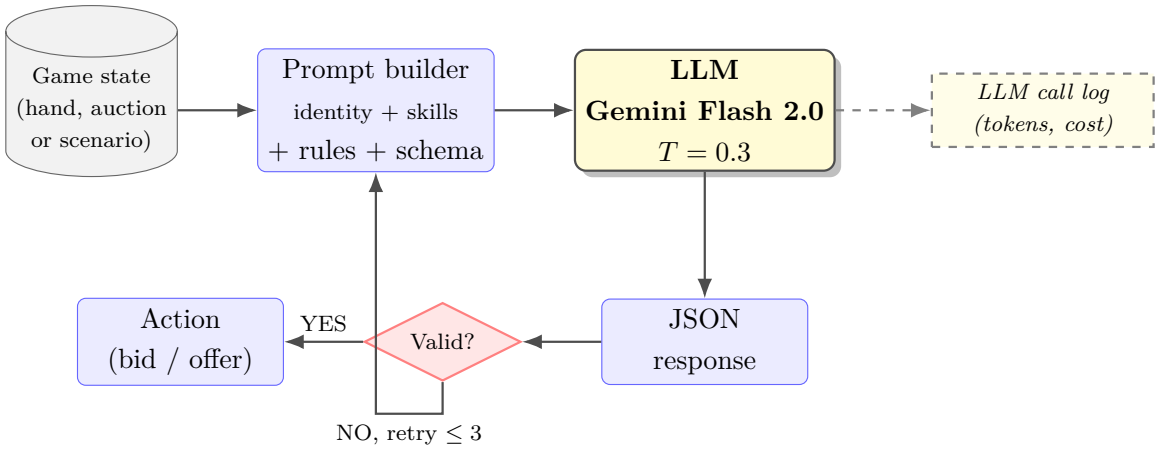


Figure 4.3: LLM agent architecture: game state → prompt builder → Gemini → JSON response → validator → action. Invalid responses trigger a retry with the validation error in the next turn. Every call is logged with token counts and cost for reproducibility.

4.3.1 System prompt anatomy

Every agent's system prompt has the same five-block structure. We formalise this so that the four profile agents differ only in the content of blocks 1 and 2:

Listing 4.1: Five-block system prompt template. Filled blocks become the production prompt in `shared/prompts.py`.

```
[BLOCK 1 - IDENTITY]
You are a <profile_name>. <one-line role description>.

[BLOCK 2 - CORE SKILLS]
Your 5-7 characteristic skills (extracted from real bridge hands):
  - <skill 1>
  - <skill 2>
  ...

[BLOCK 3 - DECISION RULES]
Domain rules (Laws of Duplicate Bridge / negotiation conventions).

[BLOCK 4 - OUTPUT FORMAT]
Respond ONLY with this JSON schema:
{ "decision": "...", "reasoning": "...", "confidence": 0.0-1.0 }

[BLOCK 5 - FEW-SHOT EXAMPLES]
Example 1: <hand> -> <bid> (reasoning: ...)
Example 2: <scenario> -> <offer> (reasoning: ...)
```

4.3.2 Temperature sampling

LLM token selection follows a softmax over logits, modulated by a temperature parameter T:

$$P(\text{token} = t \mid \text{context}) = \frac{\exp(z_t/T)}{\sum_{t'=1}^{V} \exp(z_{t'}/T)} \tag{4.12}$$

Parameters:

$P(t \mid \text{ctx})$ probability that the model emits token t next, given context.

z_t raw logit of token t produced by the model's final linear layer.

V vocabulary size (Gemini's tokeniser: $\approx$ 256K tokens).

T temperature parameter, $T > 0$. As $T \to 0$ the distribution becomes peaked (deterministic, argmax); as $T \to \infty$ it becomes uniform.

Choices for NegoPlay:

- $T = 0.3$ for bridge bidding (consistency is critical: the same hand should produce the same bid).
- $T = 0.7$ for negotiation (diversity of offers is part of the experiment).
- $T = 0.1$ for the Bridge-Expert Validation skill (we want deterministic expert opinions).

4.3.3 Per-call cost accounting

Each LLM call writes a JSON-Lines record to `results/llm_logs/`. The cost is computed as

$$C_{\text{call}} = \frac{T_{\text{in}} \cdot c_{\text{in}} + T_{\text{out}} \cdot c_{\text{out}}}{10^6} \tag{4.13}$$

Parameters:

C_{call} cost of a single LLM call, in US dollars.

T_{in} number of input tokens (system prompt + user message), reported by the provider's API metadata.

T_{out} number of output tokens (completion).

c_{in} unit cost per 10^6 input tokens (\$0.075 for Gemini Flash 2.0).

c_{out} unit cost per 10^6 output tokens (\$0.30 for Gemini Flash 2.0).

The cumulative cost is monitored against a hard cap of \$50, with an alert at \$20.

4.4 Stage 4: Dual Simulation and Alignment Measure

The core empirical claim of NegoPlay is measured by a single number: the Spearman rank correlation between profile win-rates in bridge and in negotiation.

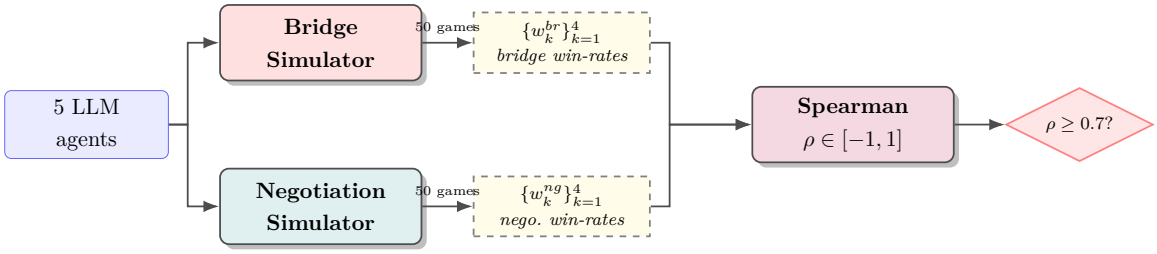


Figure 4.4: Stage 4 dual simulation: the same agents play bridge and negotiation, producing two vectors of profile-level win-rates. Spearman correlation between them is the main outcome.

4.4.1 Per-profile win rate

$$w_k^D = \frac{1}{N_D} \sum_{g=1}^{N_D} \mathbf{1}[\text{agent}_{P_k} \text{ won game } g] \tag{4.14}$$

Parameters:

w_k^D win-rate of profile k in domain $D \in \{\text{bridge}, \text{negotiation}\}$, $w_k^D \in [0, 1]$.

N_D number of games in domain D ($N_{\text{bridge}} = N_{\text{negotiation}} = 50$ in Phase 1).

$\mathbf{1}[\cdot]$ indicator function: 1 if the inner condition is true, 0 otherwise.

P_k one of the four behavioural profiles ($k \in \{1, \ldots, 4\}$), excluding the Generalist baseline.

4.4.2 Spearman rank correlation

The main outcome measure of NegoPlay. Let r_k^{br} and r_k^{ng} be the ranks of profile k within its domain (rank 1 = highest win-rate). The Spearman correlation, when there are no ties, is

$$\rho_s = 1 - \frac{6 \sum_{k=1}^{K} d_k^2}{K(K^2 - 1)}, \qquad d_k = r_k^{br} - r_k^{ng} \tag{4.15}$$

Parameters:

ρ_s Spearman rank correlation, $\rho_s \in [-1, +1]$. Value $+1$ means perfect rank agreement; -1 means perfect rank inversion; 0 means no correlation.

K number of profiles compared ($K = 4$).

d_k difference in ranks of profile k between bridge and negotiation.

The general formula (which handles ties correctly) is the Pearson correlation applied to the ranks:

$$\rho_s = \frac{\sum_k (r_k^{br} - \bar{r}^{br})(r_k^{ng} - \bar{r}^{ng})}{\sqrt{\sum_k (r_k^{br} - \bar{r}^{br})^2 \, \sum_k (r_k^{ng} - \bar{r}^{ng})^2}} \tag{4.16}$$

Parameters:

r_k^{br}, r_k^{ng} ranks of profile k in bridge and negotiation respectively.

$\bar{r}^{br}, \bar{r}^{ng}$ mean ranks (always $(K + 1)/2 = 2.5$ for $K = 4$).

Decision rule: $\rho_s \geq 0.70 \Rightarrow$ support for H3 (alignment hypothesis); $0.50 \leq \rho_s < 0.70 \Rightarrow$ partial support; $\rho_s < 0.50 \Rightarrow$ null result. Significance is tested by a two-sided permutation

test with 10,000 random permutations of the negotiation win-rate vector. Implemented as `scipy.stats.spearmanr`.

4.4.3 Falsifiability test (Phase 3)

To rule out the *tautological alignment* confound (*i.e.*, the LLM merely following the prompt rather than transferring a learned style), we run an inverse-prompt control:

$$\rho_{\text{inverse}} = \text{Spearman}(\{w_k^{br}\}_{k=1}^{K},\ \{w_k^{ng,\text{inverse}}\}_{k=1}^{K}) \tag{4.17}$$

Parameters:

$w_k^{ng,\text{inverse}}$ negotiation win-rate of an agent prompted with the *opposite* profile (e.g., a Slam Hunter in bridge becomes an Insurance-style agent in negotiation).

ρ_{inverse} alignment under the inverted-prompt control.

Falsifiability criterion: if $\rho_{\text{inverse}} \geq 0.70$, the original alignment is a prompt-following artefact, not a true behavioural transfer, and H3 must be rejected even when the main ρ_s is above threshold.

5 Schedule

The project runs for five weeks (ten sessions, two per week). Each week corresponds to a sprint with one major deliverable.

Table 5.1: Five-week sprint plan.

Sprint	**Week**	**Theme**	**Deliverable**
1	1	Research & Setup	PRD, literature review, repo skeleton
2	2	Stage 1 + Stage 2	Validated clusters, named profiles
3	3	Stage 3	Four working LLM agents
4	4	Stage 4	Bridge + negotiation simulations + alignment
5	5	Polish & Defense	Final report, presentation, demo video

6 Work Packages

The project is decomposed into seven work packages (WP). Each WP groups related tasks across one or more sessions.

WP1 – Research foundation

Sessions: 1–2. **Deliverables:** PRD v1.0, literature review (17 papers + 9 PhD-context), repository structure, two anchor papers identified.
Tasks: (T1.1) project scope alignment with supervisor; (T1.2) PRD draft and freeze; (T1.3) literature search and anchor selection; (T1.4) GitHub repository scaffold; (T1.5) Antigravity + Gemini setup; (T1.6) related-work survey of eight bridge-AI repositories.

WP2 – Data pipeline (already in place at project start)

Sessions: pre-project (Jan–May 2026). **Deliverable:** `all_matches_full.csv` – 149,208 rows.
Tasks: EuroBridge HTML scraping; card backfill via BoardAcross; player-name backfill (99.9 % coverage). *This WP is closed; the report documents it but no further work is required for NegoPlay.*

WP3 – Stage 1: clustering

Sessions: 3–4. **Deliverables:** `player_features.csv`, `player_clusters.csv`, silhouette plot, validation report.
Tasks: (T3.1) `shared/data_loader.py`; (T3.2) `stage1_clustering/features.py`; (T3.3) `stage1_clusterin` (K-Means + HDBSCAN); (T3.4) `stage1_clustering/validation.py` (Silhouette, Davies-Bouldin, permutation p-value); (T3.5) exploratory notebook with PCA / t-SNE plots.

WP4 – Stage 2: skill extraction

Sessions: 4. **Deliverables:** `skill_profiles.json`, `docs/profile_descriptions.md`.
Tasks: (T4.1) `shared/llm_client.py` (unified Gemini/Claude/OpenAI wrapper); (T4.2) game chunker; (T4.3) Gemini-based skill extractor with strict JSON schema; (T4.4) cross-chunk aggregator; (T4.5) manual profile naming by the author.

WP5 – Stage 3: agents

Sessions: 5–6. **Deliverables:** `BridgeAgent`, `NegotiationAgent`, full prompt book.
Tasks: (T5.1) `BaseAgent` abstract class; (T5.2) `BridgeAgent.make_bid`; (T5.3) `NegotiationAgent.respond`; (T5.4) prompt library in `shared/prompts.py`; (T5.5) manual sanity tests confirming behavioral variance $> 30\,\%$.

WP6 – Stage 4: simulation and alignment

Sessions: 7–8. **Deliverables:** 50 bridge games, 48 negotiation sessions, alignment report (*the key result*).
Tasks: (T6.1) `bridge_game.py` runner; (T6.2) `negotiation.py` runner; (T6.3) outcome scoring; (T6.4) optional BEN-engine adapter; (T6.5) `alignment.py` – Spearman correlation + permutation test; (T6.6) final scatter plot.

WP7 – Polish and defense

Sessions: 9–10. **Deliverables:** business-plan section, slide deck, promo video, defense rehearsal.
Tasks: (T7.1) business-plan section (TAM/SAM/SOM, token economics); (T7.2) 12–15 slide deck; (T7.3) 60–90 second promo video; (T7.4) README polish; (T7.5) Q&A document.

7 Gantt Chart and Dependencies

7.1 WP × Session matrix

Following the structure recommended by Dr. Yoram Segal, Table 7.1 maps every work-package task onto the ten project sessions. Coloured cells indicate the session in which a given task is active. Critical (blocking) tasks are coloured in dark orange; non-blocking tasks in light blue.

Table 7.1: Work-package matrix. Each row is one task; columns are the ten project sessions. Dark orange = critical / blocking. Light blue = non-blocking.

Task	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
WP1 – Research foundation										
T1.1 Scope alignment										
T1.2 PRD freeze										
T1.3 Literature review										
T1.4 Repo scaffold										
T1.5 Antigravity + Gemini setup										
T1.6 Related-work survey										
WP3 – Stage 1: clustering										
T3.1 Data loader										
T3.2 Player features										
T3.3 K-Means + HDBSCAN										
T3.4 Validation & significance										
T3.5 EDA notebook (PCA/t-SNE)										
WP4 – Stage 2: skills										
T4.1 Unified LLM client										
T4.2 Game chunker										
T4.3 Skill extractor (Gemini)										
T4.4 Aggregator										
T4.5 Profile naming										
WP5 – Stage 3: agents										
T5.1 BaseAgent										
T5.2 BridgeAgent										
T5.3 NegotiationAgent										
T5.4 Prompt library										
T5.5 Sanity tests										
WP6 – Stage 4: simulation										
T6.1 Bridge runner										
T6.2 Negotiation runner										
T6.3 Outcome scoring										
T6.4 BEN adapter (optional)										
T6.5 Alignment + significance										
T6.6 Final scatter plot										
WP7 – Polish & defense										
T7.1 Business-plan section										
T7.2 Slide deck										
T7.3 Promo video										
T7.4 README polish										
T7.5 Q&A rehearsal										

7.2 Dependencies and the critical path

The critical path is sequential: **WP1** → **WP3** → **WP4** → **WP5** → **WP6** → **WP7**. Each downstream WP depends on the output artefact of the previous one:

- WP3 cannot start before WP1 freezes the PRD (the feature set must be aligned with the research question).
- WP4 needs the clusters from WP3 to chunk games per profile.
- WP5 needs the named profiles from WP4 to build prompts.
- WP6 needs the agents from WP5 to run simulations.
- WP7 needs the alignment result from WP6 to write the conclusions.

Non-blocking (parallelisable) tasks are coloured in light blue in Table 7.1. These include the exploratory notebook (T3.5), the BEN-engine adapter (T6.4), the README polish (T7.4) and the promo video (T7.3). They can slip a session without delaying the critical path.

7.3 High-level Gantt

Table 7.2: High-level Gantt by sprint.

Work Package	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5
WP1 Research foundation					
WP3 Clustering					
WP4 Skills					
WP5 Agents					
WP6 Simulation					
WP7 Polish & defense					

8 Deliverables and Success Criteria

8.1 How will we know we succeeded?

The success of NegoPlay is defined by three concrete numbers (Table 8.1). All three come from the same code base and from runs that are logged with random seeds for full reproducibility.

Table 8.1: Research key performance indicators.

Metric	**Target**	**How measured**
Silhouette score (Stage 1)	≥ 0.4	Computed on the chosen k; visualized in `results/stage1_silhouette.png`.
Per-profile behavioral variance (Stage 3)	$> 15\,\%$	Variance of bridge win-rate across the four agents in identical 50-game tournament.
Cross-domain Spearman alignment (Stage 4c)	≥ 0.70	Correlation of per-profile win-rate ranks between bridge and negotiation.

8.2 Deliverables

Software

- A public GitHub repository `NegoPlay/` with a single SDK entry point (`src/sdk.py`).
- Four reusable modules: `stage1_clustering`, `stage2_skills`, `stage3_agents`, `stage4_simulate`.
- A unified LLM wrapper (`shared/llm_client.py`) supporting Gemini, Claude and OpenAI.

Data artefacts

- `data/processed/player_features.csv` (1,572 rows $\times$ 5 features).
- `data/processed/player_clusters.csv` (with cluster labels and centroids).
- `data/processed/skill_profiles.json` (4 named profiles, 5–7 skills each).

Reports and figures

- `results/stage1_silhouette.png` – the cluster validity plot.
- `results/bridge_winrates.csv` and `results/negotiation_winrates.csv`.
- `results/alignment_analysis.png` – the *headline* scatter plot.
- `results/alignment_report.md` – the final research answer.

Course deliverables

- This preliminary report (PDF, LaTeX source).
- Slide deck (12–15 slides).
- Promotional video (60–90 seconds).
- Business-plan section (TAM/SAM/SOM + token economics).

8.3 Engineering quality gates

- **Test coverage:** $\geq 80\,\%$ on Stage 1 (statistics must be reproducible); $\geq 60\,\%$ overall.
- **Cost cap:** hard cap of \$50 across all LLM providers, alert at \$20, target $<$ \$15.
- **Reproducibility:** the full pipeline must run end-to-end on a clean clone with one command.
- **Logging:** every LLM call writes provider, model, tokens, latency and cost to `results/llm_logs/`.

9 Risk Management

9.1 Risk register

Table 9.1: Top project risks and mitigations.

#	Risk	Lik.	Imp.	Mitigation
R1	Gemini Flash 2.0 produces inconsistent JSON or weak reasoning for agents.	Med	High	Lower temperature to 0.3; enforce `response_mime_type=application/json`; if still poor, route the affected stage to Claude / GPT through `llm_client.py`.
R2	Clusters are not statistically separable (Silhouette < 0.4).	Low	High	Pre-screen features in EDA; try alternative feature sets including BridgeHand2Vec embeddings (Sztyber-Betley et al., 2023); fall back to fewer clusters ($k = 3$).
R3	Cross-domain alignment falls below the 0.7 target.	Med	Med	Frame any $\rho < 0.5$ as a valid null result; report partial-support discussion if $0.5 \leq \rho < 0.7$; bridge $\rightarrow$ negotiation transfer is still an empirical contribution either way.
R4	LLM agents produce illegal bridge bids or invalid negotiation moves.	Med	High	Add a validator before each action; on rejection, re-prompt with the error in the next turn; cap retries at 3 to avoid infinite loops.
R5	Gemini API rate-limits (15 req/min on free tier) block Stage 4.	Med	Med	Add exponential backoff via `tenacity`; pre-batch calls; upgrade to paid tier ($\sim$\$5) if needed.
R6	Cumulative LLM cost exceeds the \$50 hard cap.	Low	High	Per-call cost log in `results/llm_logs/`; alert at \$20; pause and review prompts before exceeding the cap.
R7	The 5-week timeline is too tight (course + simultaneous PhD work).	High	High	Strict scope discipline: drop LSTM stretch goal first, then promo-video polish; keep critical path intact.
R8	Antigravity IDE bugs (preview, agents) cause work loss.	Low	Med	Commit on every session close; mirror to local VS Code; keep a backup branch tagged at the end of each WP.
R9	Player-name mismatches across competitions distort cluster sizes.	Low	Med	Use fuzzy matching in `shared/data_loader.py`; document any merges in `data/processed/player_name_map.csv`.
R10	Negotiation scenarios are perceived as toy / unrealistic.	High	Med	Frame the report explicitly as a "proof of concept in simulated environment"; cite this as future work for the PhD; ensure scenarios are based on textbook negotiation structures (M&A, JV, SaaS,

9.2 Risk monitoring

A short **risk review** is held at the end of every sprint (i.e. every two sessions). The risk register is updated in version control, and any new risk that emerges mid-sprint is added immediately. Risks R7 (timeline) and R3 (alignment outcome) are monitored continuously, since both can change project framing late in the schedule.

10 Summary

10.1 What this report covers

This preliminary report defines NegoPlay – a five-week project to discover decision-making profiles from 149,208 elite bridge tournament hands and to test, via LLM agents, whether the same profiles predict behavior in business negotiation simulations. It anchors the work in two published papers (Talwadker et al., 2022 for clustering; Rong et al., 2019 for the inference/decision split) and a theoretical foundation (Lockett et al., 2007). It specifies the dataset, the four-stage methodology, the seven work packages, the WP $\times$ session matrix, the critical path, the deliverables, and ten concrete project risks with mitigations.

10.2 Expected contribution

If the cross-domain alignment hypothesis is supported ($\rho \geq 0.7$), NegoPlay will be the first publicly reproducible demonstration that strategic profiles discovered in a clean game-theoretic environment (bridge) measurably predict behavior in a different, less structured domain (business negotiation). This contributes to the long-running PhD research thread *From Bridge Tables to Global Markets* at LUT University.

If the hypothesis is not supported, the project still contributes: (i) a validated set of bridge decision profiles, useful in its own right for bridge analytics and coaching; (ii) a unified, provider-neutral LLM client; and (iii) a clean methodological template for cross-domain behavioral transfer studies.

10.3 Beyond the course: link to the PhD

NegoPlay is the empirical baseline for Paper 1 of the author's PhD thesis. The four discovered profiles, the prompt library and the alignment methodology are designed to be reused in PhD Year 2 with: a larger dataset (BBO Vugraph adds millions of deals), a deeper model (the ENN+PNN+PPO stack described by Rong et al., 2019 and Kita et al., 2024), and real negotiation transcripts (Year 2–3). This continuity between the course project and the PhD is intentional: the same single-SDK code base will carry the work forward.

A Reproducibility Package

This appendix collects every operational detail required to reproduce the numerical results in this report. A reader with the public dataset and the code repository should be able to regenerate Tables and Figures bit-for-bit.

A.1 Code repository

- **URL:** https://github.com/annafr2/bridge-business-research
- **Subdirectory:** `projects/negoplay/`
- **License:** MIT
- **Pinned commit for this report:** see `git log -1` at time of submission.

A.2 Dataset

- **Source:** European Bridge League (EBL) public results database (https://db.eurobridge.org).
- **Competitions scraped:** Budapest 2016, Ostend 2018, Madeira 2022, Herning 2024, Poznan 2025.
- **Master file:** `data/processed/all_matches_full.csv` (149,208 rows × 48 columns; UTF-8 with BOM).
- **Scraper:** `src/downloaders/eurobridge_bulk_scraper.py` (custom Python, `requests` + `BeautifulSoup4`). Logged to `logs/scrape_log.csv`.
- **Data coverage:** player names (N/S/E/W per room) 99.9%; card holdings 54%; bidding sequences 31% (Herning + Poznan only; 100% in those competitions).

A.3 Software environment

- **OS:** Windows 11 + WSL2 (Ubuntu 24.04).
- **Python:** 3.12.10 in `venv`; full pin in `requirements.txt`.
- **Key library versions:** `scikit-learn 1.8.0`, `scipy 1.17.1`, `pandas 3.0.3`, `numpy 2.4.6`, `matplotlib 3.10.9`, `openpyxl 3.1.5`.

A.4 Stage 1 hyper-parameters

Table A.1: Stage 1 hyper-parameters (production pipeline, May 2026).

Parameter	**Value**	**Equation / function**
Minimum declared boards per player	50	filter on $n_i^{declared}$
Minimum bidding boards per player	50	filter on $n_i^{bidding}$
Variance filter: minimum CV	0.10	Eq. (4.2)
Correlation filter: maximum $\lvert r \rvert$	0.70	Pearson on z-scores
Mahalanobis α (audit only)	0.01	Eq. (4.10)
PCA target cumulative variance	0.80	Eq. (4.5)
Extreme-percentile cutoff	top 10%	per axis
Binomial significance level	$p < 0.05$	Eq. (4.11)
Random seed	42	`random_state`

A.5 Population baselines (Stage 1 output)

Population baselines are the means computed across the 563 qualifying players and are used both as the binomial-test null (p_0 in Eq. 4.11) and as the radar-chart reference. They are stored in `src/shared/bridge_validator.py`.

Table A.2: NegoPlay population baselines and assigned-profile rates (v2.1, May 2026).

Profile	**Metric**	**Baseline p_0**	**Profile mean**	**Ratio**
Slam Hunter	`slam_rate`	0.055	0.101	1.84×
Insurance Player	`partscore_rate`	0.570	0.684	1.20×
NT Specialist	`nt_rate`	0.282	0.385	1.36×
Fighter	`penalty_double_rate`	0.085	0.131	1.55×

A.6 Stage 2 – LLM call configuration

Listing A.1: Skill extraction call configuration. Identical configuration is used for all four profiles; only the user prompt changes.

```
provider = "gemini"
model    = "gemini-2.0-flash-exp"
temperature = 0.3
response_mime_type = "application/json"
max_output_tokens = 2048
system_instruction = (
    "You are a bridge analyst. Given 20-30 declared boards from a "
    "single elite player, return their 5-7 characteristic skills as "
    "structured JSON. Skills must be bridge-specific phrases, not "
    "generic personality words."
)
seed = 42
```

A.7 Stage 3 – Agent system prompts (template)

The full prompt library lives in `src/shared/prompts.py`. Listing A.2 shows the abstract template; the four profile instances differ only in the identity line and the skills block.

Listing A.2: Bridge-agent system prompt template (abridged for length).

```
You are a {profile_name} bridge bidder.

Your decision-making is characterised by:
  {skill_1}
  {skill_2}
  ...
  {skill_n}

Bridge rules you must obey:
  - Bids strictly ascend (e.g., 1NT < 2C < 2D < ... < 7NT).
  - Double (X) is legal only over an opponent's contract bid.
  - Pass is always legal.

Given the current state, return JSON with this schema:
{
  "bid": "<one of: Pass, X, XX, 1C..7NT>",
  "reasoning": "<2-3 sentences>",
  "confidence": <float between 0.0 and 1.0>
}

Example 1:
  Hand: S:AKQJxx H:Kxx D:Qx C:Ax
  Auction: -
  -> { "bid": "1S", "reasoning": "Standard opening with 17 HCP and 6S.", "
      confidence": 0.95 }

Now decide for:
  Hand: {hand}
  Auction so far: {auction}
```

A.8 Stage 4 – Simulation protocol

- **Bridge games:** 50 randomly generated PBN deals per matchup. Each agent receives only its own 13 cards. The simulator records the full auction, final contract, declarer, tricks, and IMP score relative to a fixed datum table.
- **Negotiation games:** four scenarios (M&A, JV equity, B2B SaaS pricing, government tender), each run 12–13 times per matchup for 50 games total. Maximum 10 rounds per scenario; explicit walk-away condition logged.
- **Logging:** every game writes a structured record to `results/` including provider, model, full prompt, full response, tokens, latency, cost, and outcome.

- **Random seeds:** the deal generator and the negotiation initial-state seed are derived from a master seed of 42, fixed in `src/sdk.py`.

A.9 Anchor papers

In addition to the in-text citations, the project rests on two anchor papers and one theoretical foundation:

- Anchor 1 (Stage 1, clustering): Talwadker et al. (2022).
- Anchor 2 (Stage 3, agent split): Rong et al. (2019).
- Theoretical foundation (cardinal opponent profiles): Lockett et al. (2007).

A.10 How to reproduce, end-to-end

1. `git clone https://github.com/annafr2/bridge-business-research.git`
2. `cd bridge-business-research/projects/negoplay`
3. `python3.12 -m venv .venv && source .venv/bin/activate`
4. `pip install -r requirements.txt`
5. `cp .env.example .env` – add your `GOOGLE_API_KEY`.
6. `python -m src.stage1_clustering` – regenerates `player_profiles.csv` (ML only, no API cost).
7. `python notebooks/visualize_profiles.py` – regenerates all 5 figures in `docs/images/`.
8. `python notebooks/preprocessing_comparison.py` – reruns the 5-configuration preprocessing audit.
9. `python -m src.stage2_skills` – runs Gemini skill extraction (Stage 2 onward; incurs API cost ∼$0.05).
10. `python -m src.stage4_simulate --bridge-games 50 --negotiation-games 50`.
11. `python -m src.report` – produces `results/alignment_report.md`.

A.11 Tests and continuous validation

The pipeline is covered by 73 `pytest` unit tests:

- `tests/test_features.py` – 18 tests for feature engineering.
- `tests/test_extreme_profiles.py` – 11 tests for profile assignment (including the small-sample-demotion test that locks in the Nezer sample-size correction).

- `tests/test_bridge_validator.py` – 29 tests for the bridge-expert validation skill (all LLM calls mocked).
- Run all: `pytest tests/ -v`.

Bibliography

Noam Brown and Tuomas Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456): 885–890, 2019.

Tomer James, Avi Rosenfeld, and Sarit Kraus. Predicting behavior in cheap-talk repeated games via attitude vectors. *Autonomous Agents and Multi-Agent Systems*, 2021.

Andrzej Jarosz. Reinforcement learning in contract bridge: A literature review. *Artificial Intelligence Review*, 2022.

Kenji Kita et al. A simple reproducible baseline for contract bridge bidding. *arXiv preprint arXiv:2409.12345*, 2024.

Alan J. Lockett, Charles L. Chen, and Risto Miikkulainen. Evolving explicit opponent models in game playing. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO)*, pages 2106–2113, 2007.

lorserker. BEN: Bridge engine with neural networks. https://github.com/lorserker/ben, 2023. GPL-3.0 license.

Jiang Rong, Tao Qin, and Bo An. Competitive bridge bidding with deep neural networks. *arXiv preprint arXiv:1903.00900*, 2019.

Julian Schrittwieser et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588, 2020.

David Silver et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Anna Sztyber-Betley et al. BridgeHand2Vec: Bridge hand representation. *arXiv preprint arXiv:2310.18681*, 2023.

Rukma Talwadker, Surajit Pareek, Tridib Mukherjee, and Deepak Madhok. CognitionNet: A collaborative neural network for play style discovery in online skill gaming platform. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2022. doi: 10.1145/3534678.3539114.

Hong Wang et al. Inference-decision reinforcement learning with GAN for bridge bidding. *arXiv preprint*, 2024.

Chih-Kuan Yeh, Cheng-Yu Lin, and Hsuan-Tien Lin. Automatic bridge bidding using deep reinforcement learning. *IEEE Transactions on Games*, 2018.